Experiment 13

Case Study on Limitations of Low-Code Platforms for a Complex Enterprise Use Case

Name: Divoshini V | **Register Number:** 241401020

Aim

To study the limitations of low-code platforms when they are applied to complex enterprise-level applications and to understand when pro-code extensions or hybrid approaches may become necessary.

Introduction

Low-code platforms allow applications to be developed mainly through visual development tools instead of writing large amounts of traditional program code. Applications are assembled using predefined components such as forms, reports and dashboards, and behaviour is defined through workflows, rules and configuration. A small amount of scripting is usually available for specific needs. This approach reduces development time and effort and makes it possible for a wider range of people to take part in building business applications.

Low-code platforms are well suited to standard business applications. As the requirements of an enterprise grow in terms of users, data volume, process complexity, integrations, security and customisation, the built-in capabilities of a platform may no longer be sufficient on their own. This case study examines such limitations through an enterprise vehicle service scenario, which extends the Vehicle Service & Repair Request Management application built in Zoho Creator using the Service_Request_Form and Service_Request_Report.

Case Study: Enterprise Vehicle Service Management System

A large automobile service company operating many service centres requires a system to manage its service operations, starting from the online Service_Request_Form and extending to bay scheduling, technician assignment, estimate approval, repair tracking and delivery support. The Zoho Creator application captures the owner's name, e-mail, phone, pickup address, vehicle registration number, vehicle type, odometer reading, service type, add-on services, issue description, preferred service date and drop-off time, and the owner's authorization. At enterprise scale the system may need to serve the following users and handle the following functions:

Category	Items
Users	Vehicle owners, front-office staff, service advisors, technicians, parts / inventory managers and administrators
Data and operations	Service requests, owner and vehicle details, bay and technician scheduling, and vehicle service history
Supporting functions	Email / SMS notifications, service estimate and payment collection, and spare-parts availability tracking
Enterprise integration and insight	CRM integration, dealer management system / ERP and parts-inventory integration, analytics and reporting

Figure 1 shows the architecture of this system on a low-code platform. The architecture is arranged as six groups that are connected from left to right:

241401020

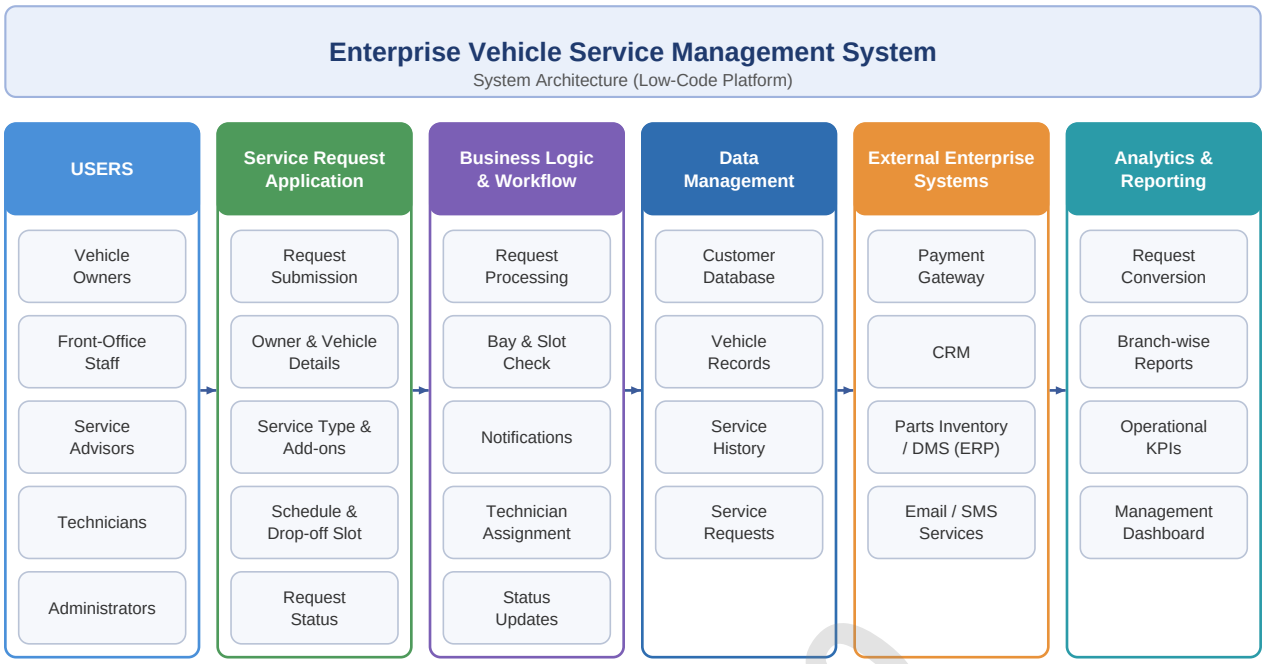

Figure 1: Enterprise Vehicle Service Management System Architecture

- **Users:** vehicle owners, front-office staff, service advisors, technicians and administrators, who interact with the system.
- **Service Request Application:** the user-facing part, covering request submission, owner and vehicle details, service type and add-ons, scheduling of the drop-off slot and request status.
- **Business Logic & Workflow:** request processing, bay and slot check, notifications, technician assignment and status updates, which move a request through the process.
- **Data Management:** the customer database, vehicle records, service history and service requests, in which the information is stored.
- **External Enterprise Systems:** the payment gateway, CRM, parts inventory / dealer management system (ERP) and email / SMS services with which the application must exchange information.
- **Analytics & Reporting:** request conversion, branch-wise reports, operational KPIs and a management dashboard, which present the stored and processed data to decision makers.

The groups interact in sequence. Users enter and view information through the application; the business logic and workflow layer processes this information; the data layer stores and supplies it; external systems provide or receive related information such as payments, owner relationships and spare-parts availability; and the analytics layer reports on the results.

How a Low-Code Platform Can Support the System

A large part of the system consists of standard application functionality that low-code platforms are designed to handle:

Capability	Use in the Vehicle Service System
Forms	Capturing owner details, vehicle details, odometer reading and issue description through the Service_Request_Form.
Reports	Viewing service requests, filtering by Service Type and Preferred Service Date, and technician assignment lists through the Service_Request_Report.

Capability	Use in the Vehicle Service System
Workflows	Moving a request through submitted, scheduled, in-service, estimate approval and delivered stages.
Business process automation	Applying rules and automatic actions, such as sending an acknowledgement email when a request is submitted.
Data management	Storing owner, vehicle and service records and performing standard create, read, update and delete operations.
User interfaces	Providing web and mobile screens for owners, front-office staff, advisors, technicians and administrators.
Notifications	Informing owners about request status, estimates and service slots through email or similar messages.
Basic integrations	Connecting to common services through built-in connectors or standard interfaces.

Limitations at Enterprise Scale

As the system grows into a complex enterprise application, the following limitations may become significant. They are discussed individually below.

Performance and Scalability

During peak seasons such as festive periods and pre-monsoon servicing drives, the number of service requests, vehicle records and simultaneous users across many branches can become very large. Complex queries and reports over large amounts of data can respond slowly, and the platform may impose limits on how far an application can be scaled. Because the underlying infrastructure is managed by the platform, the options for tuning performance are limited.

Complex Business Logic

Highly specialised business rules, such as optimal service-bay and technician scheduling, estimate and pricing calculation based on vehicle model and parts, warranty and annual maintenance contract eligibility, and technician load balancing, may be difficult to express using visual workflows and standard scripting. Such requirements may need custom development.

System Integration

Connecting the application to ERP, dealer management and CRM systems, payment gateways, SMS services, parts-supplier catalogues and legacy workshop systems can be challenging. Built-in connectors may not cover every system or data format, and complex or real-time integrations may need custom interfaces.

Customization

Standard low-code components provide common screens and layouts. Highly specialised interfaces and user experiences, such as multilingual owner portals, tailored technician job-card dashboards, or functionality that is not offered by the platform, may exceed what these standard components can provide.

2114010020

Security and Governance

Service records contain personal details of owners, including contact numbers, addresses and vehicle registration numbers, as well as payment information. Enterprises require detailed access control, governance, compliance with data-protection requirements and strong security practices. Role- and permission-based controls offered by a platform may need to be supplemented to meet fine-grained, regulatory or organisation-specific requirements, and governance of many applications must be managed carefully.

AI and Advanced Technology

Specialised artificial intelligence and machine-learning models, such as predictive maintenance from odometer and service history, diagnosing issues from the owner's description, or answering owner queries through a chatbot, and other advanced processing may require external services or custom code, since a low-code platform may not provide such capabilities directly.

Vendor / Platform Dependency

The application depends on the capabilities, restrictions, pricing and supported integrations of the selected platform. Features may be limited by the plan or edition, and moving an application to another platform later can be difficult.

Data Management

Large datasets collected over several years of vehicle servicing, complex relationships between owners, vehicles, service requests and parts, and advanced data processing, such as heavy batch processing or complex data transformations, may be difficult to handle within the data model and tools provided by the platform.

Figure 2 summarises how the limitations arise. A basic low-code application, characterised by quick development, minimal configuration and suitability for small and medium needs, faces growing enterprise requirements: more branches, users and data, complex service workflows, integration with multiple systems and higher security and compliance needs. These lead to enterprise-scale challenges such as increased complexity, higher performance demands, greater integration needs, stronger security and governance, and more customisation and flexibility. The diagram then identifies eight key limitations, which are the ones discussed above.

24149120

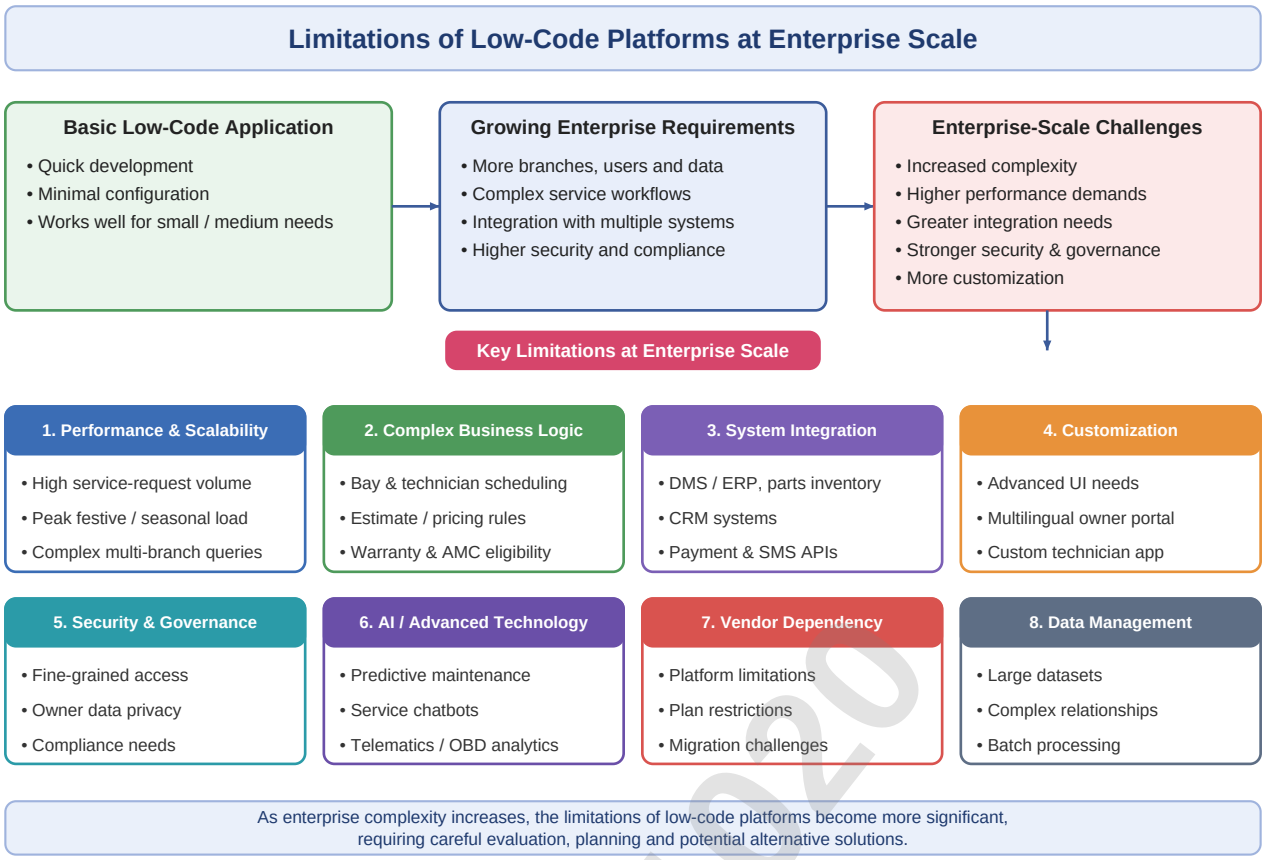

Figure 2: Limitations of Low-Code Platforms at Enterprise Scale

The relevance of each limitation to the Enterprise Vehicle Service Management System is given below.

Limitation	Relevance to the Case Study
Performance & Scalability	Growing numbers of service requests during peak seasons, and reports over service history across branches.
Complex Business Logic	Bay allocation, technician assignment, estimate calculation and warranty rules that become more complex over time.
System Integration	Payment gateway, CRM, ERP / dealer management and parts inventory, and email / SMS services.
Customization	Specialised interfaces for technicians and management, and a tailored owner experience.
Security & Governance	Different access needs for owners, front-office staff, technicians and administrators; protection of personal and payment-related data.
AI / Advanced Technology	Possible intelligent features, such as predictive maintenance and chatbots, built on service history and odometer data.
Vendor Dependency	Reliance on the platform for all application, workflow and reporting features, and on its plan limits.
Data Management	Large volumes of owner, vehicle and service records across years, with relationships to parts and technicians.

Low-Code vs Pro-Code Requirements

The suitability of low-code or pro-code development depends on the requirements of the application. Neither approach is universally better; the comparison below indicates the typical role of each.

Area	Low-Code Approach	When Pro-Code May Be Required
User interface	Standard screens, forms and layouts built from predefined components.	Highly specialised interfaces or custom user experiences beyond standard components.
Business logic	Visual workflows, rules and simple scripting for common business processes.	Complex rules and advanced processing that cannot be expressed in visual tools.
Data management	Built-in data model with forms, relationships and standard create, read, update and delete operations.	Very large datasets, complex relationships and advanced data processing.
Integrations	Built-in connectors and standard interfaces for common services.	Legacy systems, specialised or real-time integrations and systems without connectors.
Automation	Rules, scheduled actions and workflow-based automation.	Complex or high-volume automation and specialised processing.
AI/ML	Basic features offered by the platform, where available.	Custom AI models, machine-learning pipelines and specialised analytics.
Security	Role- and permission-based access control provided by the platform.	Fine-grained access control, specific compliance requirements and enterprise security needs beyond platform features.
Scalability	Suitable where the platform handles the expected users and data volumes.	High data volumes, large user numbers and demanding performance requirements.
Custom functionality	Functionality available as platform features.	Functionality that the platform does not provide.

Hybrid Approach

Organisations can combine low-code development with pro-code extensions. Low-code is used to develop standard application functionality quickly, while custom code, APIs, integrations or specialised processing are used for requirements that exceed the built-in capabilities of the platform. The structure consists of three layers:

Low-Code Application → Pro-Code Extensions → Enterprise Systems

Figure 3 shows this architecture. Users such as vehicle owners, front-office staff, service advisors, technicians and administrators access the system at the top, and the three layers are described below.

24140120

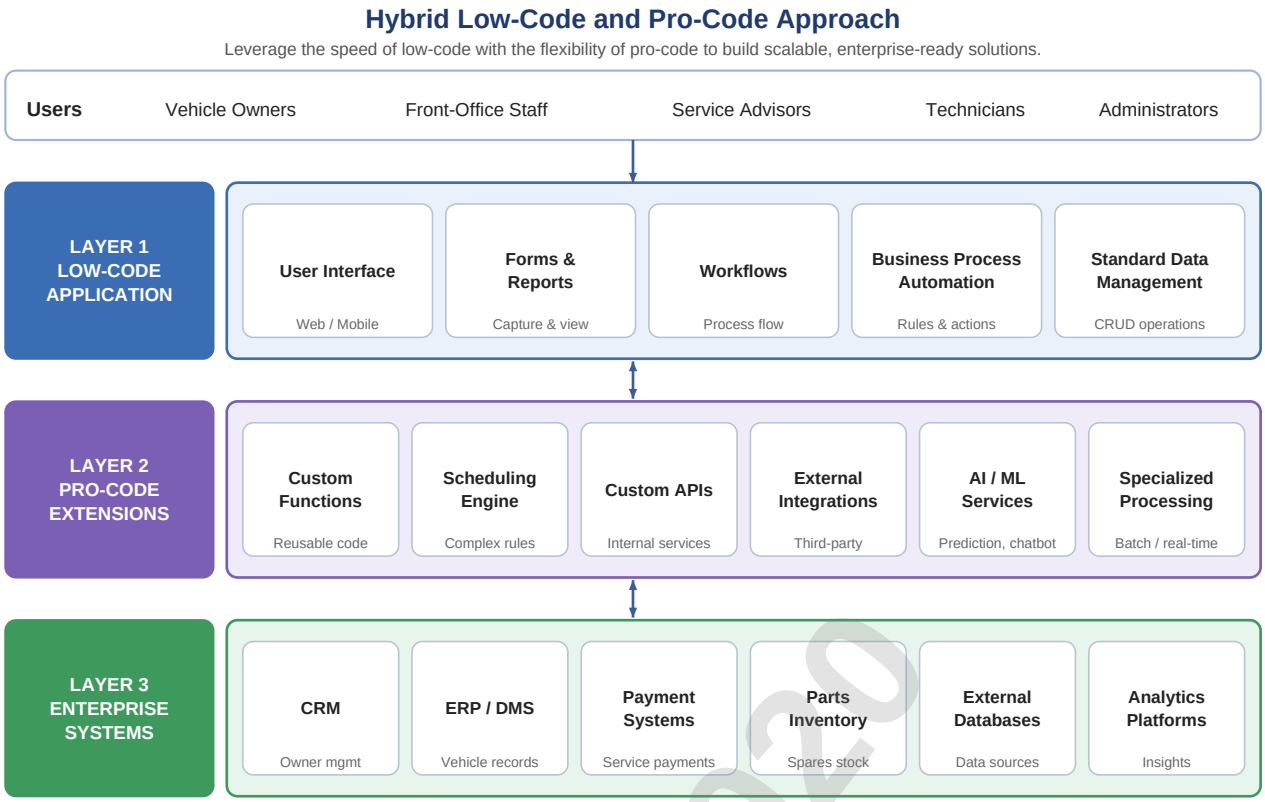

Figure 3: Hybrid Low-Code and Pro-Code Enterprise Architecture

Layer	Role
Layer 1 – Low-Code Application	Provides rapid development for standard application needs: user interface (web / mobile), forms and reports, workflows, business process automation and standard data management.
Layer 2 – Pro-Code Extensions	Adds custom logic and specialised capabilities: custom functions, a scheduling engine for complex bay and technician rules, custom APIs, external integrations, AI / machine-learning services and specialised processing.
Layer 3 – Enterprise Systems	Core systems that provide data, services and infrastructure: CRM, ERP / dealer management system, payment systems, parts inventory, external databases and analytics platforms.

The diagram also lists the reasons for a hybrid approach: faster application development, flexible customisation, integration with existing systems, support for complex business logic and the ability to extend beyond platform limitations. The arrows between the layers indicate that information flows in both directions between the application, its extensions and the enterprise systems.

When Pro-Code Extension May Become Necessary

- **Complex business logic:** when business rules and processing, such as bay scheduling, technician allocation and estimate calculation, cannot be represented using standard workflows and rules.
- **Specialized integrations:** when the application must connect to legacy, ERP, dealer management or other systems for which suitable connectors are not available.
- **Custom APIs:** when internal services or interfaces have to be exposed or consumed in a way that the platform does not support directly.

24IA01020

- **Advanced AI/ML:** when intelligent features such as predictive maintenance require custom models or machine-learning pipelines.
- **High-performance processing:** when large batches of data or real-time processing exceed what the platform can handle efficiently.
- **Specialized user interfaces:** when the required user experience goes beyond standard components.
- **Complex enterprise data processing:** when large datasets and complex relationships require advanced data handling.
- **Requirements beyond platform capabilities:** when any needed functionality is simply not offered by the selected platform.

Case Study Analysis

For the Enterprise Vehicle Service Management System, a low-code platform can be used effectively to develop the standard parts of the system rapidly: forms for service request submission, owner and vehicle details, reports on service requests filtered by service type and preferred date, workflows for request processing and technician assignment, acknowledgement notifications, and basic dashboards. These features correspond to the capabilities that low-code platforms are designed to provide, and were demonstrated in the Vehicle Service & Repair Request Management application, which used all thirteen Basic Fields of the Zoho Creator Form Builder.

As the company grows, the requirements become more demanding. A large number of requests and vehicle records raises performance and data-management concerns; integration with payment, CRM and dealer management systems raises integration concerns; differing access needs and the handling of personal data of owners raise security and governance concerns; and advanced features such as predictive maintenance may need external services. In such cases, extensions, integrations or custom development may be needed in addition to the low-code application. Low-code is therefore useful for rapidly developing standard business applications, whereas increasingly complex enterprise requirements may call for a hybrid approach, and the choice should be based on the requirements of the application.

Result

The case study demonstrated the major limitations that can arise when low-code platforms are applied to complex enterprise requirements, namely performance and scalability, complex business logic, system integration, customisation, security and governance, AI and advanced technology, vendor or platform dependency, and data management. It also showed how a hybrid low-code/pro-code approach can address requirements beyond standard low-code capabilities.

Conclusion

Low-code platforms offer speed and simplicity for standard business applications, but their limitations become more significant as enterprise complexity increases. For an application such as the Enterprise Vehicle Service Management System, standard functionality can be built quickly using low-code, while demanding requirements in logic, integration, security, scale and advanced technology may need pro-code extensions. It is therefore important to understand the limitations of a platform before selecting a development approach, so that the chosen approach matches the requirements of the enterprise application.

24100120